

Deliverable 2: Metasploit Module Selection and Analysis

Prepared by: Macpherson Aghah

Course: IT316:

Date: June 12, 2026

1. Module Overview

Purpose and Functionality

During my advanced post-exploitation research, I conducted an in-depth code analysis of the exploit/multi/script/web_delivery module within the Metasploit Framework. This module represents a sophisticated, file-less delivery mechanism designed to host and deploy script-based payloads dynamically. The core purpose of the module is to establish an on-demand, lightweight HTTP or HTTPS web server managed entirely within the local framework context.

When this command is executed on a remote, compromised endpoint via an initial access mechanism, it triggers the host operating system's native administrative utilities to reach back to the hosted web server. The server intercepts this inbound web request, extracts the requested URI, and serves raw payload bytes wrapped in a specified script format. The target utility reads these bytes directly into its volatile memory space and immediately interprets them. This allows an interactive, bidirectional network session such as a Meterpreter shell to hook back to the attacker's handler without executing any code from the physical hard disk drive.

Use Case Scenario

In practical operational environments, I found that this module becomes essential immediately after achieving an initial entry vector that grants arbitrary command execution but restricts binary file uploads. Common entry scenarios include unauthenticated remote code execution (RCE) flaws in web applications, command injection bugs in network appliance interfaces, or compromised web shells.

A major operational challenge during post-exploitation is the presence of host-based Endpoint Detection and Response (EDR) agents and traditional file-on-disk antivirus scanners. If an operator attempts to upload a compiled malicious executable, the local operating system's kernel intercepts the file-write operation, triggers an immediate hash check against cloud intelligence databases, and quarantines the file before it can spawn a thread.

By utilizing the web delivery module, I was able to solve this defense challenge completely. Because the architecture targets native interpreters like PowerShell on Windows, Python on Linux, or PHP on web hosting platforms, no unapproved third-party binary file ever touches the filesystem storage blocks. The code execution remains

entirely confined within the allocated virtual memory pages of the legitimate, trusted administrative utility process, rendering static disk signature scanning completely ineffective.

2. Code Structure

Core Mixin Inheritances

The `web_delivery` module is written in Ruby and serves as an excellent demonstration of Metasploit's modular, object-oriented design framework. Rather than writing raw network socket logic or command line format parsers from scratch, the module integrates directly with the core framework kernel by inheriting capabilities from foundational library classes known as mixins. My review of the script's inclusion directives revealed two primary mixin dependencies:

1. `Msf::Exploit::Remote::HttpServer`

This mixin encapsulates the complete technical architecture required to instantiate an independent, multi-threaded web application server. By incorporating this class, the module gains abstract access to Metasploit's underlying network communication libraries, known as the Rex assembly. It enables the module to listen on specific network interfaces, negotiate TLS encryption handshakes when an operator configures SSL options, parse raw HTTP request headers, and manage stateful network socket mappings without requiring manual protocol architecture engineering.

2. `Msf::Exploit::CmdStager`

This mixin provides the parsing architecture and structural generation logic needed to build multi-platform command sequences. It abstracts the process of taking large raw payload segments and breaking them down into digestible, syntax-compliant chunks optimized for terminal command boundaries. This ensures that the generated delivery lines fit neatly within string limit thresholds imposed by different operating system shells.

Code Organization

The architecture of the module is cleanly divided into three primary zones, each handling a distinct segment of the exploitation and deployment lifecycle:

The Initialization Block: Located at the upper boundary of the source file, this block defines the module's baseline metadata and characteristics using an associative array structures. It populates essential fields such as the development tracking history, licensing frameworks, author credits, and extensive user descriptions. Crucially, it establishes the user datastore configuration options through the `register_options` function, defining what inputs are mandatory before the script can run.

The Runtime Setup Block: This zone is driven primarily by the framework's operational triggers. When an operator types the execution command into the console terminal, this segment activates to validate the configuration parameters. It queries the active payload generation engine, computes the necessary bytes for the selected implant architecture, applies localized encoding rules, and maps the network endpoints to prepare the background listener environment.

The Request Handling Block: This zone functions as an asynchronous network event controller. It sits dormant until an external system establishes an active socket connection to the designated web URI path. Once tripped, it executes callback logic to validate the connection metadata, match the request parameters, assemble an HTTP-compliant response packet, and stream the raw stager script back down the wire to the target asset.

3. Key Functionalities

To understand exactly how the module manages state and manipulates payload code strings, I conducted a deep logical analysis of its three primary programmatic methods.

The initialize Method

This method serves as the structural constructor for the Ruby class object. It invokes the parent framework's setup routines and builds the complete parameter context for the module. Within this method, the module developers utilize the `register_options` API to enforce user parameter requirements. The core variables initialized during this process include:

- **SRVHOST and SRVPORT:** These map the precise binding parameters for the hosted web app instance. The host variable can be configured to bind broadly to all local network interfaces (0.0.0.0) or specifically to an internal staging subnet IP address, while the port parameter designates the specific TCP gateway interface to reserve.
- **URIPATH:** This option controls the literal web directory string that represents the payload storage location. If left unconfigured, the constructor flag registers a null parameter, which triggers secondary logic to randomize the address space.
- **TARGET:** An integer index variable that alters the internal logic engines. It allows the module to shift its generation routines dynamically across multiple platform targets, including Python environments, native PHP interpreters, and PowerShell terminal shells.

The primer Method

The primer method functions as the post-initialization launchpad for the web service. It overrides the generic server activation routines to ensure the delivery environment is completely staged prior to opening network ports.

When executed, the method calls out to internal framework payload generation modules to extract the raw machine code instructions or script strings required for the operator's chosen implant (such as a reverse TCP Meterpreter payload). It then evaluates the active TARGET choice. If the target is a script platform like Python, it wraps the payload bytes in a zipped, base64-encoded string execution loop.

Once the payload string is successfully formatted, the primer method constructs and outputs the exact command-line string that the user must execute on the target host, printing it clearly to the console screen.

The on_request_uri Method

This method is the heart of the module's real-time network interactions. It serves as an automated callback subroutine triggered by the underlying HTTP server mixin whenever a remote client initiates a GET request targeting the specified URI. The method accepts two critical data arguments:

- **cli:** The raw object pointing directly to the network socket stream of the connecting client asset.
- **request:** A structured object containing the fully parsed HTTP request parameters, including headers, user-agent strings, and cookie values.

When called, the method executes validation checks to confirm the incoming path perfectly matches the configured payload URI. If the validation passes, the method constructs a raw HTTP response body, injects a standard 200 OK status header alongside appropriate content-type declarations (such as text/plain or application/octet-stream), appends the dynamic payload text directly into the message payload section, and uses the socket object to flush the stream down the network pipe to the target.

4. Data Flow

The lifecycle of data flowing through the web_delivery module can be traced across four explicit operational phases, moving from localized configuration inputs to memory-space execution hooks.

[Phase 1: Input] Operator sets LHOST/SRVPORT and fires 'exploit'.

|



[Phase 2: Generation] Payload bytes are fetched and wrapped in language syntax.

(Example: Base64 encoding applied for PowerShell target).



[Phase 3: Transport] Target client executes single-line download stager.

Sends HTTP GET —► [Metasploit HttpServer Mixin]



[Phase 4: Hook] on_request_uri() sends HTTP 200 OK with payload text.

Target decodes string directly in memory —► Active Session.

Phase 1: Input and Configuration Phase

The data flow path begins when the operator populates the required framework datastore variables inside the interactive interface console shell. For my validation exercises, I configured the local parameters as follows:

```
set LHOST 192.168.1.50
```

```
set LPORT 4444
```

```
set SRVPORT 8080
```

```
set TARGET 2
```

```
exploit
```

These parameters represent the raw inputs fed directly into the class constructor variables, mapping out exactly where the final session should return and where the interim web server must listen.

Phase 2: Generation and Transformation Phase

Once the execution routine is triggered, the module intercept loops retrieve the raw binary architecture definition files associated with the designated payload. The data then undergoes an immediate programmatic transformation based on the target system type:

- **Payload Retrieval:** The system extracts the raw assembly bytes or bytecode instructions for the chosen payload framework.
- **Syntax Wrapping:** The script configuration compiler builds an implementation framework around those raw bytes. For instance, if TARGET 2 is selected, the module selects the PowerShell deployment engine.

- **Encoding Optimization:** The raw payload sequence is converted into a compressed string loop and passed through a Base64 encoding pipeline. This encoding step changes the entire text payload into a simple alphanumeric character array, preventing complex symbols, quotation marks, or line breaks from disrupting syntax integrity when the command string passes across system shells.

Phase 3: Transmission and Delivery Phase

The transformed data path now splits into an outbound command string and an inbound client request connection:

- **Stager Display:** The module prints the completed single-line command stager to the console interface window.
- **Client Invocation:** The operator runs this command line on the remote target client system. The utility (such as powershell.exe) initializes its web client engine objects, bypasses local system proxy environments, and issues an outbound standard HTTP GET request targeting the server IP and port on the local loopback or routing path.
- **Server Intercept:** The hosted web application stack catches the inbound TCP connection, reads the GET request headers, and alerts the execution handler.

Phase 4: Execution and Shell Return Phase

The final data transition occurs entirely within the host memory architectures:

- **Stream Delivery:** The `on_request_uri` loop matches the path validation data, embeds the Base64-encoded payload string into the HTTP response body, and flushes the data across the open network socket interface back to the client.
- **In-Memory Decoding:** The remote workstation's administrative utility catches the incoming stream response, extracts the raw string block, decodes the Base64 alphanumeric characters back into active executable script blocks, and passes the resulting strings directly to its local evaluation statement engine.
- **Session Hook:** The newly evaluated script initializes inside memory, executes its internal communication loops, performs a standard TCP handshake dialing out to the specified listener on port 4444, and provides a fully authenticated, interactive Meterpreter management session within the Metasploit control interface.

5. Error Handling and Security Considerations

Network and Binding Error Tracking

A frequent failure point when configuring automated, software-driven network infrastructure is the occurrence of port collisions. If a separate corporate service, local diagnostic utility, or proxy agent is already bound to the specified network interface port, any attempt to launch an additional listener on that interface will cause an unhandled system exception at the operating system socket layer.

To solve this challenge, the module developers implemented a structured exception-handling design pattern. The code blocks responsible for invoking the server mixin's binding routines are wrapped entirely inside defensive Ruby begin/rescue/end blocks.

When a binding conflict triggers a `Rex::BindFailed` exception, the module intercepts the failure gracefully. Rather than allowing the fatal crash to cascade upward and terminate the main console terminal workspace, the exception handler catches the error, halts the specific module execution thread, prints an explicit alert message to the user screen indicating that the requested port interface is blocked or unavailable, and returns control smoothly to the operator command interface.

Randomized Evasion Mechanisms

From an operational security perspective, deploying static, easily predictable application infrastructure presents an immense security risk. If a web delivery stager continuously uses standard, repetitive URL paths, automated enterprise monitoring tools, network-level intrusion detection systems (IDS), and web application firewalls (WAF) can quickly identify the pattern, flag the connection, and block the asset.

To mitigate this signature detection threat, the module utilizes an automated path randomizing mechanism within its constructor flow. If the operator does not explicitly configure a custom tracking path using the `URIPATH` parameter, the module automatically calls internal framework cryptographic string generator utilities. This utility generates an entirely randomized, unpredictable string of alphanumeric characters to serve as the structural path for the hosted payload.

References

Cybersecurity and Infrastructure Security Agency. (2023). *Understanding and mitigating threat actor use of living off the land techniques and command and control mechanisms* (Mitigation Advisory AA23-144A). U.S. Department of Homeland Security. <https://www.cisa.gov/news-events/cybersecurity-advisories>

The MITRE Corporation. (2026). *Command and control tactic (TA0011) - Enterprise framework matrix*. MITRE ATT&CK. <https://attack.mitre.org/tactics/TA0011/>

Rapid7. (2025). *Metasploit payload management, multi-handler listener engines, and dynamic process migration*. Rapid7 Open Source Project Documentation. <https://docs.metasploit.com/docs/using-metasploit/basics/components.html>